AFRICAN INSTITUTE FOR MATHEMATICAL SCIENCES

AIMS Rwanda, Kigali

"Imagination is more important than knowledge." – Albert Einstein

Course

Introduction to Neural Networks

Project Report — Assignment II

Object Detection

with YOLOv5 on the Pascal VOC Dataset

Algorithm	YOLOv5 (You Only Look Once — version 5)
Dataset	Pascal VOC 2012 (20 object classes)
Framework	Python / PyTorch
Date	March 2026

Group 3 — Members

Olusola Timothy Ogundepo
Lillian Mutesi
Lucas Mirija Razafimanantsoa
Alliance Irigenera

Contents

1 Introduction **2**

2 Dataset **2**
2.1 Pascal VOC 2012 2

3 Overview of You Only Look Once (YOLO) **2**
3.1 Definitions 2
3.2 YOLO Pipeline 2

4 Model Training and Evaluation **3**
4.1 YOLO Loss Function 3
4.2 Training Procedure 3
4.3 Evaluation 3

5 Advantages and Limitations **4**
5.1 Advantages of YOLO 4
5.2 Limitations of YOLO 4
5.3 Project-specific considerations 4
5.4 Possible improvements 4

1 Introduction

Object detection is a fundamental task in computer vision that involves identifying and localizing objects within an image. In this project, we implemented the YOLOv5 algorithm to perform object detection on the Pascal VOC 2012 dataset, which contains images of 20 different object classes. The goal was to train a model that can accurately detect and classify objects in the images, and to evaluate its performance using standard metrics.

2 Dataset

2.1 Pascal VOC 2012

The Pascal VOC 2012 dataset is a standard benchmark for object detection, containing real-world images annotated across 20 object classes such as person, car, dog, and bicycle.

For this project, the dataset is used in the YOLO format, where each image is associated with a text file containing normalized bounding box annotations. This format is directly compatible with PyTorch-based YOLO implementations, requiring no additional conversion.

Since the data was stored locally, a custom helper module `YOLODataset` was used to load images and corresponding labels, ensuring proper parsing of annotations and integration with the training pipeline.

It is important to note that the dataset exhibits class imbalance, with certain classes (such as *person*) appearing significantly more frequently than others.

3 Overview of You Only Look Once (YOLO)

You Only Look Once (YOLO) is a **real-time object detection** algorithm that formulates detection as a single regression problem. Instead of applying a classifier at multiple locations, YOLO processes the entire image in one forward pass of a neural network.

3.1 Definitions

Object Detection: The task of identifying and localizing objects in an image. Localization is typically done using **bounding boxes**, which are rectangles defined by their center coordinates, width, and height.

Grid: The input image is divided into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose center lies within it.

Bounding Box: A rectangle described by (x, y, w, h), where (x, y) is the center, and (w, h) are width and height, usually normalized by the image size.

Confidence Score: Measures how likely a bounding box contains an object and how accurate the box is. It is often defined as:

$$\text{Confidence} = P(\text{object}) \times \text{IoU}$$

Intersection over Union (IoU): A metric that measures the overlap between the predicted box and the ground truth:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Class Probability: Each grid cell predicts probabilities for each object class, representing the likelihood that a detected object belongs to a given category (output)

3.2 YOLO Pipeline

- The image is divided into an $S \times S$ grid.
- Each grid cell predicts:
  - B bounding boxes,
  - Confidence scores for each box,
  - Class probabilities.
- Predictions are combined to produce final detections.

- **Non-Maximum Suppression (NMS)** is applied to remove duplicate boxes by keeping only the highest-confidence predictions.

4 Model Training and Evaluation

4.1 YOLO Loss Function

The implemented loss function follows a grid-based formulation and is computed over all anchors. Let $\hat{y}$ denote predictions and y the ground truth. For each anchor $b \in \{1, \ldots, B\}$, the prediction vector is decomposed into:

$$(\hat{x}, \hat{y}, \hat{w}, \hat{h}, \hat{c}, \hat{p}_1, \ldots, \hat{p}_C)$$

An indicator variable $obj \in \{0, 1\}$ specifies whether an object is present in a given grid cell.

- **Localization Loss:** Applied only when an object is present ($obj = 1$), with sigmoid activation on coordinates and square root transformation on width and height:

$$\mathcal{L}_{xy} = \lambda_{\text{coord}} \sum obj \left[(\sigma(\hat{x}) - x)^2 + (\sigma(\hat{y}) - y)^2\right]$$

$$\mathcal{L}_{wh} = \lambda_{\text{coord}} \sum obj \left[(\sqrt{\hat{w}} - \sqrt{w})^2 + (\sqrt{\hat{h}} - \sqrt{h})^2\right]$$

- **Confidence Loss:**

$$\mathcal{L}_{\text{obj}} = \sum obj \, (\sigma(\hat{c}) - c)^2$$

$$\mathcal{L}_{\text{noobj}} = \lambda_{\text{noobj}} \sum (1 - obj) \, \sigma(\hat{c})^2$$

- **Classification Loss:** Computed only for cells containing objects:

$$\mathcal{L}_{\text{class}} = \sum obj \sum_{k=1}^{C} (\hat{p}_k - p_k)^2$$

The total loss is obtained by summing over all anchors and averaging over the batch:

$$\mathcal{L} = \frac{1}{N} \sum_{b=1}^{B} (\mathcal{L}_{xy} + \mathcal{L}_{wh} + \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{noobj}} + \mathcal{L}_{\text{class}})$$

where λ_{coord} and λ_{noobj} are weighting parameters controlling the contribution of localization and no-object confidence terms.

4.2 Training Procedure

For computational efficiency, the model is trained on a subset of the dataset, with all images resized to a fixed resolution. The network is optimized using the Adam optimizer with a learning rate of 10^{-3} over a fixed number of epochs.

At each iteration, the ground truth annotations are transformed into the YOLO grid-based representation using a helper function `encode_target`, which maps bounding boxes and class labels into a tensor of shape $S \times S \times (B \cdot 5 + B \cdot C)$.

Here, S denotes the grid size (the image is divided into an $S \times S$ grid), B is the number of bounding boxes predicted per grid cell, and C is the number of object classes. The term 5 corresponds to the parameters (x, y, w, h, c), where (x, y) are the box center coordinates, (w, h) its width and height, and c the confidence score.

The model predictions are then compared to the encoded targets using the YOLO loss function, and parameters are updated via backpropagation.

4.3 Evaluation

The model is evaluated by computing the mean YOLO loss over different datasets. In evaluation mode, no gradient updates are performed, and the loss is averaged across all batches.

The mean loss values obtained are:

- Training subset (500 images) ≈ 12.4

- Validation set ≈ 12.7
- Augmented training subset ≈ 11.2
- Augmented Validation set ≈ 13.2

5 Advantages and Limitations

5.1 Advantages of YOLO

- **Real-time detection:** YOLO performs object detection in a single forward pass, making it significantly faster than region-based methods.
- **Global reasoning:** The model processes the entire image at once, allowing it to capture contextual information and reduce false positives.
- **End-to-end training:** Detection is formulated as a single regression problem, simplifying the pipeline compared to multi-stage approaches.

5.2 Limitations of YOLO

- **Localization errors:** The grid-based structure can lead to imprecise bounding boxes, especially for small objects.
- **Limited capacity per grid cell:** Each cell predicts a fixed number of boxes, which may cause missed detections when multiple objects are close together.
- **Sensitivity to small objects:** Objects occupying a small portion of the image are harder to detect due to spatial downsampling.

5.3 Project-specific considerations

- **Limited dataset size:** Only 500 out of 13,690 images were used, limiting generalisation and increasing overfitting risk.
- **Reduced image resolution:** Training at 128×128 leads to loss of fine details and poorer detection of small objects.
- **Computational constraints:** CPU-only training imposed a simpler architecture and limited training to 10 epochs, preventing full convergence.
- **Data augmentation issue:** Horizontal flips were not consistently applied to bounding boxes, corrupting a significant portion of labels.
- **Class imbalance:** The dominance of the *person* class biases predictions toward that category.
- **Evaluation limitation:** Performance was assessed using loss only, without standard metrics such as mAP@0.5.

5.4 Possible improvements

- **Use full dataset:** Train on all 13,690 images to improve generalisation.
- **Higher resolution + GPU:** Train at 416×416 with GPU acceleration for better detection performance.
- **Correct data augmentation:** Apply horizontal flips jointly to images and bounding boxes ($c_x \to 1 - c_x$).
- **Anchor box optimisation:** Use k-means clustering on ground-truth boxes to improve localisation.
- **Learning rate scheduling:** Use strategies such as cosine annealing to stabilise training.
- **Better evaluation metrics:** Report mAP@0.5 for standardised performance comparison.
- **Handle class imbalance:** Use focal loss to reduce bias toward dominant classes.
- **Transfer learning:** Initialise from a pretrained backbone to improve convergence and accuracy.